Systems-Level Detection and Supply Chain Intelligence: A Comprehensive Evaluation of Open-Source SBOM, CBOM, and AIBOM Tooling

The software supply chain has irrevocably transformed from a static collection of proprietary binaries and explicit open-source libraries into a highly dynamic, globally distributed ecosystem of interconnected components. Modern digital infrastructure integrates pre-compiled external code, complex cryptographic algorithms transitioning to post-quantum standards, and non-deterministic artificial intelligence models powered by autonomous agents. As a result of this escalating complexity, the regulatory and operational mandate for transparency has expanded exponentially. Global directives—such as the United States Executive Order 14028, the European Union Cyber Resilience Act (CRA), the EU AI Act, and the National Security Memorandum on Post-Quantum Cryptography (NSM-10)—now demand machine-readable, verifiable, and continuously updated inventories of all system components[1].

To satisfy these stringent requirements, the concept of the Bill of Materials (BOM) has bifurcated into highly specialized, interoperable domains. The foundational Software Bill of Materials (SBOM) tracks application dependencies and structural composition; the Cryptography Bill of Materials (CBOM) inventories cryptographic assets, keys, and algorithms to facilitate post-quantum migration; and the Artificial Intelligence Bill of Materials (AIBOM or ML-BOM) catalogs machine learning models, training data provenance, hyperparameter configurations, and autonomous agentic behaviors. This report provides an exhaustive, peer-level evaluation of the leading open-source detection tools across these three critical domains, focusing specifically on their system-level identification capabilities, underlying architectural methodologies, and downstream reporting fidelity.

1. Software Bill of Materials (SBOM) Generation and Analysis

A Software Bill of Materials serves as the definitive, machine-readable inventory of libraries, frameworks, direct dependencies, transitive dependencies, and their exact cryptographic hashes within a given product[5]. In the contemporary open-source ecosystem, automated generation tools are an absolute necessity, as modern applications possess hundreds of transitive dependencies that render manual tracking statistically impossible[7]. However, the efficacy of an SBOM relies entirely on the detection depth of the generator, the chosen output specification, and the tool's ability to integrate into continuous integration and continuous deployment (CI/CD) pipelines.

1.1 The Standardization Landscape: SPDX and CycloneDX

The reporting capabilities of any SBOM tool are inherently constrained by the specifications it supports. The industry has largely standardized on two formats: CycloneDX and SPDX. CycloneDX, maintained by the OWASP Foundation and standardized as ECMA-424, was architected from inception with security, vulnerability management, and component reachability as its primary objectives[6]. The CycloneDX specification has aggressively expanded over the past several iterations, with version 1.6 formally introducing native cryptographic asset tracking and version 1.7 finalizing machine learning model metadata, data provenance, and citations[4].

Conversely, the Software Package Data Exchange (SPDX), governed by the Linux Foundation and recognized internationally as ISO/IEC 5962:2021, originated primarily in the licensing and intellectual property compliance space[6]. The release of SPDX 3.0 represents a monumental structural shift, transitioning from a flat document model to a modular profile system. An SPDX 3.0 document is constructed around a mandatory Core Model—which dictates provenance and document creation mechanics—stacked with optional namespaces such as the Software Profile, the Security Profile, the Build Profile, and newly introduced profiles explicitly designed for AI and Datasets[11]. The Security Profile in SPDX 3.0 drastically enhances reporting capabilities by embedding Vulnerability Exploitability eXchange (VEX) status justifications, CVSS assessments across multiple versions, and Exploit Prediction Scoring System (EPSS) metrics directly into the artifact[11]. Both formats have reached a level of maturity where they seamlessly support downstream supply chain risk management, provided the generation tool accurately populates the schema.

1.2 Deep Binary Inspection and Container Ecosystems: Syft vs. Trivy

For system-level scanning—particularly concerning the introspection of container images, raw filesystems, and compiled build artifacts—the open-source ecosystem is dominated by two primary utilities: Syft and Trivy. While often compared directly, their architectural philosophies differ significantly.

Syft, developed by Anchore under an Apache 2.0 license, is a purpose-built SBOM generator[14]. Its singular focus yields the deepest cataloger coverage currently available in the open-source tier. Syft executes comprehensive layer-by-layer analysis of container images and supports over thirty diverse package ecosystems, ranging from operating-system-level package managers like APK, DEB, and RPM to language-specific environments[7]. Syft's system-level identification superiority manifests in its ability to parse complex edge cases, such as extracting dependency data from nested Java "uber-jars" or identifying statically linked libraries compiled directly into Rust and Go binaries[7]. Syft operates entirely offline and generates artifacts in CycloneDX, SPDX, and a proprietary JSON format, pairing natively with Grype—Anchore's companion open-source vulnerability scanner—to enable an architecture where SBOM generation and vulnerability correlation are handled as distinct, cacheable pipeline stages[1].

Trivy, maintained by Aqua Security, originated as a vulnerability scanner and has evolved into an all-in-one security toolkit that generates SBOMs alongside misconfiguration detection and secret scanning capabilities[1]. Trivy's primary advantage is its operational breadth; a single binary execution can target container registries, local filesystems, virtual machine images, and

live Kubernetes clusters[1]. For organizations seeking to consolidate their toolchain, Trivy provides rapid, holistic triage. However, because its SBOM generation mechanisms are secondary to its vulnerability scanning engine, its dependency cataloger depth occasionally trails Syft when analyzing deeply nested or obfuscated compiled binaries[15]. Regarding reporting capabilities, Trivy is highly versatile, exporting data in CycloneDX, SPDX, SARIF (Static Analysis Results Interchange Format) for IDE and CI/CD integration, and native GitHub Dependency Snapshots, while natively supporting in-toto attestations via Cosign[15].

1.3 Reachability Analysis and Intermediate Representations: cdxgen

A chronic limitation of traditional SBOM scanners is the generation of excessive false-positive vulnerability alerts. Scanners reliably identify vulnerable dependencies, but fail to determine if the host application's execution path ever invokes the vulnerable function. The OWASP CycloneDX Generator (cdxgen) directly addresses this limitation through the implementation of advanced data-flow and reachability analysis[18].

cdxgen is a polyglot generator supporting over twenty programming languages, focusing exclusively on producing high-fidelity CycloneDX documents[8]. Its system-level identification capabilities are exponentially enhanced by its integration with atom, a novel intermediate representation (IR) toolkit powered by the chen (Code Hierarchy Exploration Net) library[19]. When executed in "deep mode" (via flags such as --with-reachables), cdxgen performs static slicing of the application codebase to trace variable assignments and function calls from designated entry points down to external library sinks[18].

This process employs algorithms like Framework-Forward Reachability (FFR) and Semantic Reachability—the latter of which consumes OpenAPI specification files to map HTTP endpoint routes directly to code hotspots[22]. If a vulnerability exists in a library but the callstack evidence proves the code is unreachable, security analysts can definitively deprioritize the alert[18]. This callstack evidence is serialized directly into the resulting CycloneDX document[18]. Furthermore, cdxgen expands its system-level reporting by offering a tracebom command to dynamically profile running applications, capturing negotiated TLS cipher suites and dynamically resolved software components via runtime analysis[8].

1.4 Enterprise Scalability and Firmware Specialization: Microsoft SBOM and UniBOM

For massive enterprise deployments involving mixed-language monorepos, the Microsoft SBOM tool provides a highly scalable, open-source generation alternative[14]. Leveraging Microsoft's internal component detection libraries, it integrates deeply into CI/CD pipelines to output SPDX format documents at build time, ensuring high-performance snapshotting of complex environments like NuGet, Go, and pip[14].

Conversely, the Internet of Things (IoT) and embedded firmware ecosystems present unique challenges, as legacy C and C++ codebases routinely lack unified package managers, relying instead on ad-hoc source inclusion or pre-compiled static libraries[23]. The UniBOM project addresses this gap by performing multi-layered binary analysis on router firmware and

embedded operating systems[25]. UniBOM integrates string literal scanning, data compression similarity detection, and binary delta computations to identify third-party code cloning without relying on package manifests[23]. Crucially, UniBOM incorporates an AI-based classification engine to evaluate detected firmware components against memory safety constraints, linking identified Common Platform Enumerations (CPEs) to specific memory vulnerabilities (such as CWE-476 NULL pointer dereferences or CWE-787 Out-of-bounds writes), thereby establishing a precise risk profile for headless edge devices[25].

Feature / Metric	Syft	Trivy	cdxgen	UniBOM
Architectural Focus	Deep dependency cataloging	Unified vulnerability scanning	Polyglot reachability analysis	Firmware & binary code cloning
Output Standard	CycloneDX, SPDX, Custom JSON	CycloneDX, SPDX, SARIF	CycloneDX (Versions 1.4 - 1.7)	CycloneDX
Reachability Evidence	None	None	Yes (via atom and chen IR)	Yes (Memory safety focus)
Target Workloads	Containers, archives, source	K8s, VMs, containers, repos	Source code, containers, APIs	IoT devices, embedded routers
Vulnerability Integration	Separate tool required (Grype)	Built-in continuous scanning	Threat modeling & attack vectors	CPE/CWE AI-based classification

1.5 SBOM Post-Processing: Merging, Augmentation, and Attestation

System-level identification rarely ends at the generation phase; pipeline orchestration requires sophisticated post-processing tools. The open-source manifest-cli utility allows engineering teams to programmatically merge multiple SBOMs of the same format into a unified system inventory, which is critical when complex applications are built across multiple discrete stages or microservices[27]. It natively wraps generators like Syft, Trivy, and spdx-sbom-generator, allowing snapshot modes that assign semantic tags to distinct production environments[27]. Furthermore, the sbomify-action toolkit provides essential augmentation and enrichment functions[29]. While a raw scanner might only extract a package name and version, sbomify queries authoritative external registries—such as PyPI, crates.io, and ClearlyDefined—to enrich

the BOM with lifecycle dates, support periods, and rigorous license data[29]. It injects critical organizational metadata, such as supplier information and manufacturer details, ensuring the final artifact meets the strict NTIA minimum elements and CISA regulatory requirements before the SBOM is cryptographically signed and stored as an OCI artifact in the container registry[29].

2. Cryptography Bill of Materials (CBOM) and Post-Quantum Agility

The impending obsolescence of classical asymmetric cryptography, driven by rapid advancements in quantum computing, has established a firm deadline for global infrastructure modernization. Regulatory bodies are moving aggressively; the National Institute of Standards and Technology (NIST) intends to deprecate RSA and elliptic-curve cryptography algorithms after 2030, and the NSA's CNSA 2.0 guidelines mandate a full transition to post-quantum cryptography (PQC) by 2033[31]. The fundamental barrier to this migration is visibility. Organizations cannot refactor cryptographic dependencies if they do not know where these algorithms are deployed, how they are parameterized, or which network services actively depend on them[32].

The Cryptography Bill of Materials (CBOM) standardizes this inventory process. A comprehensive CBOM provides a machine-readable record of all cryptographic algorithms (symmetric, asymmetric, hash functions, key derivation functions), the software libraries implementing them (e.g., OpenSSL, Bouncy Castle), the digital certificates securing communications, and the exact protocols governing their usage[31].

2.1 The CBOM Specification and Regulatory Drivers

The structural foundation for the CBOM was formally adopted into the OWASP CycloneDX specification beginning with version 1.6 and significantly expanded in 1.7, establishing an internationally recognized schema (ECMA-424) for cryptographic asset reporting[9]. The specification extends the CycloneDX object model by introducing a cryptographic-asset component type, under which algorithms, protocols, certificates, and related cryptographic material (such as keys and tokens) are categorized[9]. This modularity allows a CBOM to exist as a standalone document or to be seamlessly integrated via BOM-Link relationships into a broader application SBOM[35].

2.2 The PQCA CBOMkit Architecture: A Multi-Layered Approach

The most authoritative open-source implementation of the CBOM specification is the **CBOMkit** suite, developed originally by IBM Research and subsequently donated to the Linux Foundation's Post-Quantum Cryptography Alliance (PQCA)[36]. The architecture of CBOMkit demonstrates a profound understanding of system-level identification, segmenting the discovery process across five interconnected components.

For static codebase analysis, the suite relies on the sonar-cryptography plugin (Codename: Hyperion)[36]. Rather than relying on simple regular expression matching, this engine operates on the Abstract Syntax Trees (ASTs) generated by language-specific parsers for Java, Python, and

Go, ensuring a highly accurate identification of cryptographic API calls[36]. This engine is wrapped by cbomkit-lib for standalone programmatic access and cbomkit-action for automated execution within CI/CD pipelines[36].
However, the true system-level identification capabilities reside within **cbomkit-theia** (Theia). This Go-based command-line tool bypasses source code entirely, focusing instead on inspecting compiled container images (Docker, OCI, Singularity) and raw directories[36].
cbomkit-theia operates through an extensible, multi-plugin architecture:

- **Certificate Plugin:** Extracts and parses X.509 certificates from the filesystem, injecting signature algorithms and public key metrics into the CycloneDX schema[38].
- **Java Security Plugin:** Locates Java runtime configuration files (e.g., java.security) and analyzes properties like jdk.tls.disabledAlgorithms. By comparing these configurations against the cryptographic algorithms detected in the container, it computes an empirical "confidence level" representing the likelihood that a specific algorithm is actively executable in that specific environment[38].
- **OpenSSL Configuration Plugin:** Parses openssl.cnf structures to catalog globally permitted TLS protocol versions and cipher suites[38].
- **Secrets Plugin:** Leverages the gitleaks engine to uncover exposed private keys and secret tokens embedded within the infrastructure[38].

The outputs from these scanners are centralized into the **Mnemosyne** database, utilizing a Command Query Responsibility Segregation (CQRS) pattern to manage scan state and query retrieval[36]. The data is visualized through the **Coeus** frontend and continuously evaluated for quantum safety by the **Themis** compliance engine. Themis utilizes Open Policy Agent (OPA) and Rego language policies to automatically classify assets against a whitelist of approved quantum-safe Object Identifiers (OIDs), outputting detailed compliance violation reports[36].

2.3 Deep Linux Introspection and Embedded Systems: CipherIQ cbom-generator

While the PQCA suite is highly optimized for containerized cloud workloads, the **CipherIQ cbom-generator** delivers unmatched system-level cryptographic discovery for bare-metal Linux servers, edge computing nodes, and embedded Linux builds[3]. Dual-licensed under GPL-3.0 and developed in C11, CipherIQ supports a massive array of distributions, including Ubuntu, RHEL, Alpine, and highly constrained architectures like Yocto and OpenWrt[3].
CipherIQ's architectural differentiator is its instantiation of a strict four-level dependency relationship graph: SERVICE -> PROTOCOL -> CIPHER_SUITE -> ALGORITHM[3]. This paradigm provides complete traceability; if a vulnerable algorithm is detected, an infrastructure engineer can trace the exact dependency path back to a specific high-level service (e.g., Nginx, strongSwan, Mosquitto) responsible for invoking it[3].
CipherIQ achieves this mapping through exhaustive system-level identification methodologies:

- **Dynamic ELF Header Parsing:** Instead of executing potentially untrusted target binaries, the scanner safely parses VERNEED and SONAME headers directly from Executable and Linkable Format (ELF) files within the target root filesystem. This enables reliable

cross-architecture scanning, allowing an x86_64 host to audit an ARM64 Yocto image accurately[3].

- **Active Service Contextualization:** CipherIQ extracts TLS and SSH configurations from active daemons by combining process table analysis, open port inspection, and systemd service file parsing. It is sophisticated enough to detect OpenSSH post-quantum hybrid key exchange methods (e.g., sntrup761x25519-sha512@openssh.com)[3].
- **Key Lifecycle Tracking:** The system safely catalogs seven distinct key formats (PEM, DER, PKCS#8, OpenSSH, etc.) and ten key types, verifying lifecycle states against NIST SP 800-57 guidelines to flag weak configurations, such as RSA keys under 2048 bits[3].

Furthermore, CipherIQ embeds advanced reporting capabilities by directly appending PQC risk assessments into the CycloneDX 1.6/1.7 properties metadata. It evaluates 48 NIST-finalized OIDs, calculates estimated quantum break-years, and categorizes every asset as SAFE, TRANSITIONAL, DEPRECATED, or UNSAFE, automatically generating a phased migration roadmap aligned with FIPS 203/204/205 standards[3].

2.4 Integration with Software Scanners: cdxgen and SCANOSS

CBOM generation is increasingly becoming a native feature within advanced SBOM generators. For instance, cdxgen leverages its intermediate representation capabilities to inventory Java keystores and perform source-level cryptographic tracking for JavaScript and TypeScript ecosystems[8]. Similarly, the **SCANOSS crypto-finder** analyzes source code to produce an interim JSON file rich with matched code snippets, which is subsequently converted into a validated CycloneDX 1.6 CBOM. SCANOSS implements rules-based deduplication, ensuring that if multiple heuristics identify the same cryptographic primitive or padding scheme, the system consolidates the findings into a single, highly detailed component entry[40].

Feature / Metric	PQCA cbomkit-theia	CipherIQ cbom-generator	SCANOSS crypto-finder
Primary Environment	Cloud Containers, Filesystems	Bare-metal Linux, Yocto, Edge	Local Source Code
Identification Logic	Environment Configs, Executability	Dynamic ELF & Process Mapping	Code Snippet Rule Matching
Relationship Mapping	Flat list with confidence scoring	Deep 4-tier Service-to-Algo graph	File-level snippet metadata
Reporting Standard	CycloneDX 1.6	CycloneDX 1.6 & 1.7	CycloneDX 1.6

PQC Risk Prioritization	External via Themis Rego policies	Internal assessment & break-years	Custom downstream analysis

4. Artificial Intelligence Bill of Materials (AIBOM) and Agentic Governance

The integration of foundational large language models (LLMs), fine-tuned predictive networks, and autonomous multi-agent systems has fundamentally altered software economics. However, AI introduces non-deterministic behaviors, proprietary data dependencies, and novel attack vectors—such as prompt injection, data poisoning, and model evasion[4]. The Artificial Intelligence Bill of Materials (AIBOM) provides the necessary structural transparency to operationalize AI securely and achieve compliance with frameworks like the EU AI Act and the NIST AI Risk Management Framework (RMF)[2].

4.1 Deconstructing the AI System: A Multi-Layered Taxonomy

A functional AIBOM extends far beyond simple software dependency listing. According to industry consensus frameworks, a complete AIBOM must capture metadata across several distinct layers[41]. The *Data Layer* tracks the origin, licensing, and preprocessing steps of both training datasets and real-time inference data streams[41]. The *Model Layer* inventories the foundation models (e.g., OpenAI, Anthropic), fine-tuned variants, internal architectures, and specific deployment hyperparameters[41]. The *Dependency and Infrastructure Layers* log the underlying software frameworks (TensorFlow, PyTorch) alongside the hardware compute resources (GPUs, TPUs) required for execution[2]. Finally, the *Security and Governance Layers* map access paths, identity entitlements, and ownership structures to establish an audit trail[4].

4.2 AIBOM Standardization: SPDX 3.0 Profiles and CycloneDX ML-BOM

The implementation of these layers relies heavily on emerging standard schemas. The CycloneDX specification introduced robust ML-BOM capabilities starting in version 1.5, reaching maturity in version 1.7[4]. The CycloneDX approach favors integrating machine learning components tightly within the existing software dependency graph, adding specific capabilities to track data lineage, performance metrics, and compliance citations natively[4].
Alternatively, the SPDX 3.0 specification introduced dedicated namespaces explicitly designed to handle complex AI workloads[11]. The SPDX 3.0 *AI Profile* requires comprehensive safety and ethical disclosures, instituting properties such as AI/autonomyType to document a system's operational independence, AI/energyConsumption to measure the environmental impact of training runs, and AI/safetyRiskAssessmentType to categorize the model into EU AI Act risk tiers (Low, Medium, High, Serious)[45]. Furthermore, properties like AI/hyperparameter provide

transparency into the pre-defined configurations that dictate learning efficiency, ensuring model reproducibility[45]. This is paired closely with the SPDX 3.0 *Dataset Profile*, which documents data noise, known biases, sensitivity indicators, and specific sensor origins[11].

4.3 Multi-Tiered Code and Container Analysis: Cisco AI BOM

Generating an AIBOM across a distributed codebase requires advanced identification logic. The **Cisco AI BOM** (aibom) is an open-source CLI utility engineered to construct a dependency graph of AI assets—including models, agents, tools, Model Context Protocol (MCP) servers, prompts, and guardrails—by scanning source code and container images[46].

Cisco AI BOM employs a highly sophisticated, three-tier detection architecture[46]:

1. **Tier 1 (Deterministic High-Confidence):** Using LibCST for deep Abstract Syntax Tree (AST) parsing in Python, and tree-sitter for languages like Java, Go, and Rust, the engine detects framework instantiations and dependencies. It compares imports against a continuously updated DuckDB knowledge base to identify AI-specific SDKs[46].
2. **Tier 2 (Cross-Reference Resolution):** The scanner maps environment variables and multi-file dependencies, correctly correlating a model invocation in a Python script back to an API key defined in a .env or docker-compose.yaml file[46].
3. **Tier 3 (Agentic LLM Classification):** To eliminate the high false-positive rates typical of static analyzers, Cisco AI BOM utilizes a local or cloud-hosted LLM agent (powered by Deep Agents and LangChain)[46]. Using "Locality-Aware Batching," the tool feeds contextually related code snippets to the LLM. The agent utilizes functions like trace_data_flow and search_package_info to definitively classify whether an arbitrary class is a standard web server or a specialized AI MCP server, pulling metadata directly from registries like Hugging Face or PyPI[46].

Cisco AI BOM supports robust container integration, autonomously extracting source code from Docker, Podman, Skopeo, or Crane containers without requiring runtime execution[46]. It leverages Anchore Syft as a fallback to generate standard SBOM metadata, enriching the final AI BOM. The tool excels in reporting capabilities, exporting to ten distinct formats including SARIF, SPDX 3.0, CycloneDX, and an interactive HTML dashboard[46].

4.4 Resolving Shadow AI via Kernel and Network Telemetry: DefendAI AgentDiscover

While static analysis is critical, it suffers from a fundamental blind spot: "Shadow AI." Developers can deploy unauthorized agents, swap LLM bindings at runtime, or alter prompt templates in local configuration files without committing changes to source control[47]. In these scenarios, static scanners will report a system as compliant while a malicious or ungoverned agent silently exfiltrates data in production[47].

The **DefendAI AgentDiscover Scanner** solves this problem by shifting identification to the operating system kernel and network layers[47]. It introduces a classification taxonomy that categorizes agents based on runtime correlation, identifying **"GHOST"** agents—workloads actively communicating with AI providers that completely lack corresponding source code

declarations or Kubernetes deployment manifests[47].
AgentDiscover achieves this visibility through a five-layer detection mechanism:

- **Layer 1 (Static Analysis):** AST pattern matching to establish a baseline inventory of declared frameworks like LangChain, AutoGen, and CrewAI[48].
- **Layer 2 (Network Heuristics & Introspection):** The scanner monitors active socket connections via psutil[49]. However, major providers like AWS Bedrock rotate endpoints across generic EC2 IPs, rendering reverse-DNS lookups ineffective. AgentDiscover utilizes **Forward-DNS correlation**, proactively caching the IP addresses of known LLM hostnames to attribute traffic accurately[50]. It then introspects the active process command line to identify the exact Python script generating the traffic, dynamically applying Layer 1 signatures to memory[50].
- **Layer 3 (Kubernetes eBPF Monitoring):** For production clusters, the scanner integrates with Cilium Tetragon, deploying extended Berkeley Packet Filter (eBPF) probes directly into the Linux kernel[48]. This zero-overhead tracing maps outbound AI API calls directly back to the exact pod, container, and execution path responsible, providing irrefutable attribution[48].
- **Layer 5 (Cloud Audit & SSE Proxies):** To bypass TLS inspection and Secure Service Edge (SSE) proxies (e.g., Zscaler ZIA, Palo Alto Prisma Access), AgentDiscover ingests cloud audit logs directly. This allows it to extract the HTTP User-Agent and caller identities, revealing specific framework attributions (e.g., boto3, langchain-aws) that local network monitoring cannot intercept[50].

4.5 Cryptographic Trust in Model Training Pipelines: AIBoMGen

For internally trained models, evaluating the resulting binary post-hoc provides no guarantee of data provenance. A malicious developer can easily falsify the dataset metadata attached to the model card[51]. **AIBoMGen**, developed by IDLab (Ghent University and imec), enforces cryptographic trust directly at the source of model creation[51].
AIBoMGen operates as a locked-down, neutral Platform as a Service (PaaS) training environment[51]. To establish an absolute root of trust, the platform intentionally prohibits remote code execution (RCE) on the worker nodes during training[51]. As the approved TensorFlow classification or regression job runs, the platform autonomously observes and captures the precise datasets, hyperparameters, resource metrics, and foundation models utilized[51].
Upon completion, AIBoMGen cryptographically hashes all input files, environment states, and output binaries. It generates a signed CycloneDX AIBOM paired with an **in-toto attestation link file**[51]. The in-toto framework generates cryptographically signed records of every supply chain action. Downstream auditors can subsequently verify the layout validity, signature authenticity, and artifact hashes against the AIBOM, proving conclusively that the model was generated using the declared dataset without external tampering[51].

4.6 Native Weight Extraction and Runtime Emission: pitloom and runtime-aibom-emitter

The ecosystem is further enriched by specialized extraction utilities. The **pitloom** project, an open-source SBOM generator for Python and AI workflows, provides deep integration with the hatchling build system[55]. pitloom is uniquely capable of programmatically extracting native metadata directly from compiled machine learning weights—specifically supporting GGUF, ONNX, and PyTorch models, as well as parsing Hugging Face repository URLs to construct transparent dependency chains for deployed models[57].

Similarly, the **runtime-aibom-emitter** bridges the gap between static analysis and dynamic execution[59]. Operating alongside the aibom-policy-engine (which utilizes OPA-style rules to enforce model pinning and deny risky data combinations), the runtime emitter ingests OpenTelemetry (OTel) trace evidence[59]. It combines the static AI inventory with real-time trace data to generate a CycloneDX-compatible AIBOM that reflects only the models, tools, and endpoints that were definitively invoked during live inference, eliminating dead code paths from compliance reports[59].

Tool / Framework	Detection Methodology	Key System Capabilities	Primary Output	Target Workload
Cisco AI BOM	AST Parsing + Agentic LLM	Resolves env vars, extracts container code	CycloneDX, SPDX 3.0, SARIF	Repos, Container Images
AgentDiscover	eBPF + Network + Cloud Logs	Correlates code to live kernel syscalls	SARIF, Console JSON	Kubernetes, Local OS
AIBoMGen	PaaS observation + Hash generation	Prevents RCE, enforces in-toto links	Signed CycloneDX	Model Training Pipelines
pitloom	Binary weight parsing	Extracts metadata from GGUF/ONNX	SPDX / CycloneDX	ML Model Files

5. System-Level Reporting, Consolidation, and Downstream Workflows

The ultimate utility of SBOMs, CBOMs, and AIBOMs lies in their ability to be consolidated, audited, and actioned across the enterprise infrastructure. Open-source tooling provides several mechanisms for transforming raw inventories into strategic intelligence.

5.1 Beyond the SBOM: OBOM, HBOM, and SaaSBOM

System-level reporting is actively expanding beyond the application layer. The cdxgen platform possesses the capability to generate an **Operations Bill of Materials (OBOM)**[18]. Targeted heavily toward SOC analysts and incident responders, an OBOM scans live Linux, Windows, or macOS hosts to inventory operating system packages, running services, and runtime configurations, which is invaluable for hardening drift reviews or detecting persistence mechanisms[8]. Furthermore, cdxgen supports the generation of a **Hardware Bill of Materials (HBOM)** for Apple Silicon and Linux hosts, as well as **SaaSBOMs** to map external cloud service dependencies, allowing platforms to merge HBOMs and OBOMs into a single topology-aware host document[8].

5.2 Vulnerability Exploitability eXchange (VEX) and Actionable Intelligence

The generation of complex BOMs inevitably leads to an overwhelming volume of vulnerability alerts, many of which are mathematically or structurally impossible to exploit in a given environment. The **Vulnerability Exploitability eXchange (VEX)** standard is crucial for mitigating this alert fatigue[5].

When cdxgen uses its atom intermediate representation to determine a vulnerable library is unreachable[18], or when the PQCA's cbomkit-theia calculates a zero confidence level for a deprecated cryptographic algorithm[38], this intelligence is serialized. Using platforms like OWASP Dependency-Track, organizations can automatically generate VEX attestations that flag the affected components as "Not Affected" with precise technical justifications[5]. This reporting mechanism allows admission controllers (such as OPA/Gatekeeper or Kyverno) to verify Cosign signatures on the SBOMs and VEX documents, securely admitting only verifiable, risk-mitigated workloads into the Kubernetes cluster[5].

6. Synthesized Conclusions and Strategic Recommendations

The software supply chain has evolved from a linear progression of code into a multi-dimensional matrix of cryptographic primitives, autonomous artificial intelligence models, and deeply nested container dependencies. The open-source tooling ecosystem has rapidly matured to address this complexity, offering specialized identification and reporting capabilities that rival or exceed commercial alternatives.

To establish a resilient, cryptographically verifiable, and compliant supply chain architecture, organizations should adopt the following strategic implementation model:

- **Establish Verifiable SBOM Foundations:** Integrate **Syft** natively into the CI/CD pipeline to generate highly accurate, layer-by-layer dependency maps of all containerized applications, utilizing sbomify-action to enrich the data to CISA compliance standards[15]. Couple this with **cdxgen** to execute deep data-flow reachability analysis, generating precise callstack evidence that significantly reduces the noise of false-positive CVE

alerts[18].

- **Accelerate Post-Quantum Migration with CBOMs:** As regulatory deadlines for quantum safety approach, deploy the **CipherIQ cbom-generator** across bare-metal and embedded Linux infrastructure to establish a dynamic, 4-tier map of cryptographic service dependencies[3]. Simultaneously, leverage the **PQCA CBOMkit** suite to monitor containerized cryptographic configurations, employing Open Policy Agent (OPA) to enforce NIST and CNSA 2.0 quantum-safe compliance baselines automatically[3].
- **Enforce Agentic Governance via AIBOMs:** Address the escalating risk of non-deterministic and Shadow AI by generating comprehensive AIBOMs using **Cisco AI BOM** during the build phase, leveraging its local LLM agent to accurately classify complex Model Context Protocol integrations and eliminate static false positives[46]. Furthermore, deploy the **DefendAI AgentDiscover Scanner** with eBPF hooks into production Kubernetes clusters. This ensures continuous runtime telemetry, attributing exact outbound LLM traffic to specific pods and immediately flagging ungoverned "Ghost" agents that attempt to bypass static security controls[47].

By treating SBOM, CBOM, and AIBOM generation not as disparate compliance checkboxes, but as a unified, continuous pipeline of system-level intelligence, organizations can proactively defend their infrastructure against the next generation of supply chain, cryptographic, and algorithmic threats.

Works cited

1. 12 Free Open-Source SCA Tools 2026: Trivy, Grype, Syft Tested - AppSec Santa, https://appsecsanta.com/sca-tools/open-source-sca-tools
2. What is an AI bill of materials (AIBOM)? - Sysdig, https://www.sysdig.com/learn-cloud-native/what-is-ai-bill-of-materials-aibom
3. CipherIQ/cbom-generator: Cryptographic asset discovery and PQC readiness scanner with CycloneDX CBOM output - GitHub, https://github.com/CipherIQ/cbom-generator
4. The Model Bill of Materials: What AI Act Auditors Will Ask For - Medium, https://medium.com/@michael.hannecke/the-model-bill-of-materials-what-ai-act-auditors-will-ask-for-8bf2da012faa
5. SBOM Tools 2026: Syft vs Trivy vs Dependency-Track Compared | NomadX, https://devsecops.ae/sbom-tools-comparison-2026/
6. Best SBOM Tools 2026: Syft Leads OSS, FOSSA Leads Commercial - AppSec Santa, https://appsecsanta.com/sca-tools/sbom-tools-comparison
7. SBOM Generation Tools Compared: Syft, Trivy, cdxgen, and More | Sbomify, https://sbomify.com/2026/01/26/sbom-generation-tools-comparison/
8. CycloneDX Generator (cdxgen) - GitHub, https://github.com/cdxgen/cdxgen
9. Advancing Cryptographic Transparency: CBOM Standardization in CycloneDX - PKI Consortium, https://pkic.org/events/2025/pqc-conference-kuala-lumpur-my/THU_B_0900_basil-hess_advancing-cryptographic-transparency-cbom-standardization-in-cyclonedx_merged.pdf

10. Overview – SPDX, https://spdx.dev/about/overview/
11. SPDX 3.0 Is Released | FOSSA Blog, https://fossa.com/blog/spdx-3-0/
12. All about SPDX 3.0. (This post was first created in... | by Interlynk | Medium, https://medium.com/@interlynkblog/all-about-spdx-3-0-7763c9e93c78
13. SPDX 3.0 data fields - Black Duck, https://documentation.blackduck.com/bundle/bd-hub-2026.1/page/Reporting/spdx3datafields.html
14. Guide to SBOM Tools: 5 Picks for Enterprise Security Teams | Wiz, https://www.wiz.io/academy/application-security/top-open-source-sbom-tools
15. SBOM Tools Compared: Syft vs Trivy vs CycloneDX CLI - Secure Pipelines, https://secure-pipelines.com/ci-cd-security/sbom-tools-compared-syft-trivy-cyclonedx-cli/
16. Grype: Free CVE Scanner for Containers (2026) | AppSec Santa, https://appsecsanta.com/grype
17. Reporting - Trivy, https://trivy.dev/docs/dev/docs/configuration/reporting/
18. CDXgen - OWASP CycloneDX SBOM Generator - AppSec Santa, https://appsecsanta.com/cdxgen
19. Introduction | Atom Project, https://appthreat-atom.readthedocs.io/
20. atom and chen join AboutCode, https://aboutcode.org/blog/atom-chen-aboutcode
21. cdxgen-docs/advanced.jsonl · CycloneDX/cdx-docs at e4783f7babbbd5414fcdc0a6832828535739b88a - Hugging Face, https://huggingface.co/datasets/CycloneDX/cdx-docs/blob/e4783f7babbbd5414fcdc0a6832828535739b88a/cdxgen-docs/advanced.jsonl
22. Reachability Analysis | OWASP dep-scan - Read the Docs, https://depscan.readthedocs.io/reachability-analysis/
23. UniBOM – A Unified SBOM Analysis and Visualisation Tool for IoT Systems and Beyond | Request PDF - ResearchGate, https://www.researchgate.net/publication/398627581_UniBOM_-_A_Unified_SBOM_Analysis_and_Visualisation_Tool_for_IoT_Systems_and_Beyond
24. Towards Understanding Third-party Library Dependency in C/C++ Ecosystem, https://www.researchgate.net/publication/363331597_Towards_Understanding_Third-party_Library_Dependency_in_CC_Ecosystem
25. [2511.22359] UniBOM -- A Unified SBOM Analysis and Visualisation Tool for IoT Systems and Beyond - arXiv, https://arxiv.org/abs/2511.22359
26. How Memory-Safe is IoT? Assessing the Impact of Memory-Protection Solutions for Securing Wireless Gateways - ResearchGate, https://www.researchgate.net/publication/385529621_How_Memory-Safe_is_IoT_Assessing_the_Impact_of_Memory-Protection_Solutions_for_Securing_Wireless_Gateways
27. cli/README.md at main · manifest-cyber/cli - GitHub, https://github.com/manifest-cyber/cli/blob/main/README.md
28. Public repository containing Manifest CLI Documentation and Releases - GitHub, https://github.com/manifest-cyber/cli
29. GitHub Action for sbomify., https://github.com/sbomify/sbomify-action

30. Integrations | Sbomify, https://sbomify.com/features/integrations/
31. CBOM: Cryptographic Bill of Materials Guide - O3 Security, https://o3.security/academy/cbom-cryptographic-bill-of-materials
32. Best CBOM & Cryptographic Discovery Tools 2026: Enterprise Comparison - QCecuring, https://www.qcecuring.com/blog/best-cbom-cryptographic-discovery-tools-2026
33. CBOM Explained: Why You Need a Cryptographic Bill of Materials - Unsung Ltd., https://www.unsungltd.com/blog-posts/cbom-explained-why-you-need-a-cryptographic-bill-of-materials
34. What Is a CBOM? Cryptographic Bill of Materials Guide - Checkmarx, https://checkmarx.com/glossary/what-is-a-cbom/
35. Cryptographic Bill of Materials (CBOM) Deep-Dive, https://postquantum.com/post-quantum/cryptographic-bill-of-materials-cbom/
36. PQCA CBOMkit Architecture – Post-Quantum Cryptography Alliance, https://pqca.org/blog/2026/pqca-cbomkit-architecture/
37. CBOMkit - Zurich - IBM, https://www.zurich.ibm.com/cbom/
38. GitHub - cbomkit/cbomkit-theia: A tool for detecting cryptographic assets in container images and directories, and generating CBOMs., https://github.com/IBM/cbomkit-theia
39. GitHub - cbomkit/cbomkit: A toolset for dealing with Cryptography Bill of Materials (CBOM), https://github.com/cbomkit/cbomkit
40. crypto-finder/docs/OUTPUT_FORMATS.md at main - GitHub, https://github.com/scanoss/crypto-finder/blob/main/docs/OUTPUT_FORMATS.md
41. AI-BOMs: A Practical Guide to AI Bills of Materials - Wiz, https://www.wiz.io/academy/ai-security/ai-bom-ai-bill-of-materials
42. What is an AI Bill of Materials (AIBOM)? - JFrog, https://jfrog.com/learn/ai-security/aibom/
43. owasp aibom, https://owaspaibom.org/
44. Complete Guide to AIBOMs - Sonatype, https://www.sonatype.com/resources/articles/aiboms
45. Understanding the SPDX 3.0 AI BOM Support | by Omar Santos | Medium, https://becomingahacker.org/understanding-the-spdx-3-0-ai-bom-support-7f3dbdd28345
46. GitHub - cisco-ai-defense/aibom: AI Bill of Materials through source code scanning, https://github.com/cisco-ai-defense/aibom
47. Defend-AI-Tech-Inc/agent-discover-scanner - GitHub, https://github.com/Defend-AI-Tech-Inc/agent-discover-scanner
48. Agent Discover Scanner – DefendAI – AgentOps Platform for AI Security, https://defendai.ai/scanner
49. Finding Ghosts: Detecting AI Agents Running in Kubernetes With No Source Code, https://dev.to/mwaseem-defendai/we-found-a-ghost-detecting-an-ai-agent-running-in-kubernetes-with-no-source-code-1ac6
50. AgentDiscover Scanner - GitHub Marketplace,

https://github.com/marketplace/actions/agentdiscover-scanner
51. AIBoMGen: Generating an AI Bill of Materials for Secure, Transparent, and Compliant Model Training - arXiv, https://arxiv.org/html/2601.05703v1
52. AIBoMGen: Generating an AI Bill of Materials for Secure, Transparent, and Compliant Model Training - arXiv, https://arxiv.org/pdf/2601.05703
53. GitHub - idlab-discover/aibomgen-cli: Go CLI tool that scans a repository for Hugging Face model usage and emits a CycloneDX AI/ML Bill of Materials., https://github.com/idlab-discover/AIBoMGen-cli
54. AIBoMGen: Generating an AI Bill of Materials for Secure, Transparent, and Compliant Model Training - ResearchGate, https://www.researchgate.net/publication/399667421_AIBoMGen_Generating_an_AI_Bill_of_Materials_for_Secure_Transparent_and_Compliant_Model_Training
55. bact/pitloom: SBOM generation for Python & AI projects ... - GitHub, https://github.com/bact/pitloom
56. hatchling · GitHub Topics, https://github.com/topics/hatchling
57. aibom · GitHub Topics, https://github.com/topics/aibom?o=desc&s=updated
58. Arthit Suriyawongkul bact - GitHub, https://github.com/bact
59. GitHub - airblackbox/runtime-aibom-emitter, https://github.com/airblackbox/runtime-aibom-emitter
60. aibom · GitHub Topics, https://github.com/topics/aibom?o=desc&s=stars
61. OWASP cdxgen, https://owasp.org/www-project-cdxgen/